01 iDeliveryFood Plano Mestre Revisado

Regras desta revisão — aplicação integral

Este documento conserva o conteúdo e os módulos do PDF anexado, com alterações nas instruções afetadas, sem resumo. As regras abaixo se aplicam a todos os módulos e critérios de aceite deste arquivo.

Mobile nativo e testes por plataforma

Android: Kotlin e Jetpack Compose; iOS: Swift e SwiftUI, projetos separados. Sem app híbrido/React. Implementar e testar cada plataforma independentemente: unitários de domínio e ViewModel, integração API, UI, deep links, sessão, permissões, offline e acessibilidade. CI Android com toolchain apropriado; CI iOS em runner macOS com Xcode. Assinatura e contas de distribuição são externas ao Git. Falta de ambiente Apple deve aparecer como pendência, nunca como build/teste concluído. Web/PWA permanece entrega web separada, jamais substitui os apps nativos.

MCP exato fornecido pelo proprietário

{
"mcpServers": {
"supabase": {
"serverUrl": "https://mcp.supabase.com/mcp?project_ref=dwsoochnfxldfvcaijie&features=docs%2Caccount%2Cdatabase%2Cdebugging%2Cdevelopment%2Cfunctions%2Cbranching%2Cfeedback"
}
}
}

SQL remoto e seed

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

GitHub depois dos testes

O Antigravity deve executar commit e push em https://github.com/JuanBMMontesinos/iDeliveryFood.git depois de testar o código e revisar diff e segredos. Verificar branch/remotes e preservar histórico; não

usar force-push, mudar visibilidade ou contornar proteção. Respeitar gates de aprovação do ambiente e autenticar por mecanismo seguro. Executar testes locais antes do push e acompanhar CI remoto depois; falha de CI requer correção e novo push, sem alegar release aprovado. Confirmar SHA remoto. Push não é deploy nem publicação nas lojas.

Idioma principal

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

Documentação e segurança preservadas

Manter documentação completa, testes unitários, integração, RLS/pgTAP, e2e e controles de cybersegurança dos módulos originais. Atualizar arquitetura, ERD, migrations aplicadas, seed, API, matriz de permissões, instruções nativas de build/teste, i18n, CI/CD, operação e evidências. Não incluir segredos; não declarar certificação de segurança sem validação independente.

Conteúdo integral revisado do documento anexado

iDeliveryFood — Plano mestre revisado

Orientações e limites

PACOTE REVISADO — PROMPTS POR APLICATIVO, PORTAL E MÓDULO

Este material substitui o pacote anterior. É uma especificação e conjunto de comandos para o agente construir o iDeliveryFood incrementalmente; não é código já implementado nem certificação de produção.

Os prompts cobrem explicitamente app web e app mobile do cliente, app mobile do entregador, portal do parceiro, gestor de pedidos, portal de redes, console logístico, área de frota, administração central e backoffices financeiro, atendimento, comercial, marketing, relatórios, risco e privacidade, além da infraestrutura comum.

Como usar: cole as instruções permanentes antes do prompt do módulo. Em uma nova sessão, repita-as e peça ao agente para ler docs/DELIVERY_STATUS.md e os contratos existentes. Execute um módulo por tarefa, respeitando dependências. Não envie todos para execução simultânea. IDs indicam módulos, não obrigatoriamente a ordem numérica.

Projeto Supabase corrigido: dwsoochnfxldfvcaijie. Executar migrations e seed no ambiente de desenvolvimento autorizado conforme esta revisão. Rotacionar a senha exposta; arquivos não contêm senha nem URI credenciada.

Não há publicação GitHub, acesso ao banco ou implantação realizada por este pacote. Contas externas, contratação de serviços, credenciais seguras, revisão jurídica e aprovações permanecem responsabilidade do proprietário.

Mapa dos aplicativos e portais

Cliente web/PWA: apps/customer-web — M05.

Cliente Android: apps/customer-android (Kotlin/Compose); cliente iOS: apps/customer-ios (Swift/SwiftUI) — M06.

Entregador Android: apps/courier-android (Kotlin/Compose); entregador iOS: apps/courier-ios (Swift/SwiftUI) — M11.

Parceiro/estabelecimento: apps/partner-web — M07.

Gestor de pedidos: área operacional do partner-web — M08.

Redes e multiunidades: área de rede no partner-web — M09.

Administração central: apps/admin-web — M16.

Despacho/logística: área restrita no admin-web — M14.

Frotas/operadores: área tenant-scoped — M15.

Backoffice financeiro: área restrita no admin-web — M13.

Atendimento: console restrito no admin-web e entradas nos apps — M17.

Comercial/contratos: área restrita no admin-web — M18.

Marketing/cupons/avaliações: área restrita no admin-web — M19.

Indicadores/risco/privacidade: áreas com papéis distintos no admin-web — M21.

Mercados/separação: extensão no partner-web — M22.

Backend/API e worker: apps/api e apps/worker, compartilhados por todos os apps. Não duplicar um backend por aplicativo.

Cada área pode ter URL/rota própria, mas compartilhar código não concede permissão: isolamento depende de API, RLS, grants e escopos.

Índice dos prompts

M00 — Auditoria, produto e contratos compartilhados

Dependências: Início obrigatório

M01 — Monorepo, ambientes e desenvolvimento local

Dependências: M00

M02 — Supabase: banco, migrations, Auth, RLS e Storage

Dependências: M00, M01; projeto dwsoochnfxldfvcaijie; executar migrations e seed no Supabase autorizado

M03 — Identidade, contas, permissões e onboarding

Dependências: M02

M04 — Catálogo, lojas, cobertura e preços

Dependências: M02, M03

M05 — App web do cliente — portal de compra/PWA

Dependências: M03, M04, M10; pagamento final depende M12

M06 — App mobile do cliente — Android e iOS

Dependências: M03, M04, M10; M12 para pagamento; contratos de M05

M07 — Portal web do parceiro — gestão do estabelecimento

Dependências: M03, M04, M10; M13 para financeiro

M08 — Gestor operacional de pedidos do restaurante

Dependências: M07, M10

M09 — Portal de redes, franquias e multiunidades

Dependências: M07, M08, M13

M10 — Pedidos, carrinho, cotação e máquina de estados

Dependências: M03, M04

M11 — App do entregador — Android e iOS

Dependências: M03, M10, M14; M13 para ganhos

M12 — Pagamentos, PSP, webhooks e estornos

Dependências: M10; escolha e credenciais seguras do PSP

M13 — Backoffice financeiro e conciliação

Dependências: M10, M12; M14 para entregas

M14 — Logística, despacho e console operacional

Dependências: M10, M03; modelo de entrega aprovado

M15 — Portal de operadores logísticos e frotas

Dependências: M11, M14, M13; contratos de frota aprovados

M16 — Administração central, acessos e cadastros

Dependências: M03, M04, M10

M17 — Atendimento, suporte e resolução de incidentes

Dependências: M10, M16; M12/M13 para reembolso

M18 — Comercial, contratos e onboarding de parceiros

Dependências: M16, M07

M19 — Marketing, cupons, avaliações e campanhas

Dependências: M04, M10, M13, M16

M20 — Notificações, tempo real e integrações externas

Dependências: M03, M10; eventos dos demais módulos

M21 — Relatórios, indicadores, risco e privacidade

Dependências: M13, M16, M17, M19

M22 — Extensão mercados, separação e substituições

Dependências: M04, M10, M12, M07; só após escopo aprovado

M23 — Cibersegurança transversal e auditoria técnica

Dependências: M01 em diante; gate antes do release

M24 — QA, testes unitários, integração, E2E e desempenho

Dependências: Cada módulo; integração final após M23

M25 — DevOps, observabilidade, deploy, backup e operação

Dependências: M01, M23, M24; ambiente/provedores aprovados

M26 — Documentação completa, integração final e GitHub

Dependências: Todos os módulos habilitados; M23-M25

Ordem recomendada e paralelismo seguro

Fase A: M00 -> M01 -> M02 -> M03. Usar projeto confirmado dwsoochnfxldfvcaijie, credenciais seguras e senha exposta rotacionada. Aplicar migrations e seed no Supabase de desenvolvimento e verificar os resultados.

Fase B: M04 -> M10. Publicar contratos revisados internamente, não necessariamente deploy.

Fase C: M16 e estruturas M07/M05/M06; M08 depois de pedidos. Frentes paralelas só com contratos e propriedade de arquivos definidos; cada agente em branch/worktree distinto, sem editar a mesma migration ou contrato simultaneamente.

Fase D: M12 -> M13; M14 -> M11; completar checkout e tracking dos clientes. Credenciais/contratos dos provedores são bloqueadores reais.

Fase E: M17, M18, M19, M20 e M21, respeitando dependências; notificações básicas M20 podem antecipar-se após M10. M09 depende de financeiro/rede. M15 e M22 apenas se aprovados.

Qualidade contínua: M23 e M24 desde o início, não apenas ao final. M25 quando infraestrutura definida. M26 encerra integração/release após gates.

Não há duração/custo estimados sem decisões de escopo, disponibilidade dos fornecedores e validação do código produzido.

Matriz administrativa proposta

Consumidor: seus endereços/pedidos/chamados, sem outros clientes.

Entregador: ofertas elegíveis e atribuições próprias, dados da rota necessários e ganhos próprios.

Proprietário parceiro: sua organização/unidades, equipe e gestão autorizada; gerente conforme unidade; operador sem bancário/financeiro; financeiro do parceiro conforme escopo.

Operações: pedidos e incidentes, sem alterar dados bancários ou executar repasse.

Logística: oferta/atribuição/ocorrência, sem acesso irrestrito a documentos/finanças.

Suporte: contexto mínimo e solicitação de ações; alçada de refund separada.

Financeiro: ledger/conciliação e ações aprovadas; segregação entre cadastro bancário e liquidação.

Comercial: leads/condições/contratos, sem movimentar dinheiro.

Marketing: campanha/cupom e métricas, sem credenciais ou documentos pessoais amplos.

Risco/cadastro: análise documental e decisões justificadas; revisão humana.

Auditoria/privacidade: leitura necessária e solicitações, sem privilégios ilimitados implícitos.

Administrador técnico: acesso controlado, MFA, auditoria e mínimo uso; não conta diária compartilhada.

Frota/rede: somente entidades autorizadas, não acesso global por serem gestores.

Entregáveis e gates

Cada módulo deve gerar código executável, contratos, migrations quando necessárias, testes, manual e evidências reais. Status por módulo: NÃO INICIADO / EM ANDAMENTO / BLOQUEADO / IMPLEMENTADO LOCAL / TESTADO / VALIDADO STAGING / LIBERADO. “Implementado” não significa “configurado” ou “pronto para produção”.

Gate de segurança: nenhum segredo, controles de tenant, RLS/grants, MFA e achados revisados. Gate

financeiro: reconciliação/idempotência/ledger e PSP contratado. Gate QA: suites e resultados reproduzíveis. Gate operação: backup/restore/alertas e responsáveis. Gate jurídico: contratos/privacidade/entregadores/tributação revisados.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

Publicação: depois dos testes, executar commit e push no repositório solicitado, confirmar SHA remoto e CI; sem force-push e respeitando gates do ambiente. O documento instrui o Antigravity a executar, não afirma que já publicou.

Fontes técnicas para o agente consultar

Estas referências foram indicadas no contexto anterior e devem ser verificadas pelo agente ao implementar. Não substituem a validação das versões e requisitos atuais dos provedores.

Google Antigravity: https://antigravity.google/

Supabase MCP: https://supabase.com/docs/guides/getting-started/mcp

Supabase RLS: https://supabase.com/docs/guides/database/postgres/row-level-security

OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

As decisões específicas de stack e metas neste documento são propostas de projeto, não alegações de certificação ou capacidade já medida.